Security management

Chapter 9 (Sections 9.02, 9.03 & 9.04) and Chapter 21 contain extensive accounts of security issues and measures used to prevent problems. For the operating system the many aspects of security management include:

- provision for recovery when data is lost
- prevention of intrusion
- ensuring data privacy.

Error detection and recovery

Errors can arise in the execution of a program either because it was badly written or because it has been supplied with inappropriate data. Other errors are associated with devices not working correctly. Whatever the cause of an error, the operating system should have the capability to interrupt a running process and provide error diagnostics where appropriate. In extreme cases, the operating system needs to be able to shut down the system in an organised fashion without loss of data.

TASK 8.01

For each of the above categories of operating system task, each point could be placed in a different category. Make an abbreviated list of these categories and add arrows to show different categories where each point could be placed.

Question 8.01

It is useful to describe the management tasks carried out by an operating system as being primarily one of the following types:

- those assisting the user of the system
- those concerned with the running of the system.

Considering the management tasks that have already been categorised, can you identify them as belonging to one or other of the above types? Are there any problems in doing this?

8.03 Utility software

A utility program can be provided by the operating system or it can be installed separately. It is a program that is not executed as part of the normal running of the operating system. Instead it is a program that the user or the operating system can decide to run when needed. For example, some utility programs manage hard disks.

Hard disk formatter and checker

A disk formatter will typically carry out the following tasks:

- removing existing data from a disk that has been used previously
- setting up the file system on the disk, based on a table of contents that allows a file recognised by the operating system to be associated with a specific physical part of the disk
- partitioning the disk into logical drives if this is required.

Another utility program, which can be a component of a disk formatter, performs disk contents analysis and, if possible, disk repair when needed. The program first checks for errors on the disk. Some errors arise from a physical defect resulting in what is called a 'bad sector'. There are a number of possible causes of bad sectors. However, they usually arise either during manufacture or from mishandling of the system. An example is moving the computer without ensuring that the disk heads are secured away from the disk surface.

Other errors arise from an event such as a loss of power or an error causing sudden system shutdown. As a result some of the files stored on the disk might no longer be useable. A disk repair utility program can mark bad sectors and ensure that the file system no longer tries to use them. When the integrity of files has been affected, the utility might be able to recover some of the data but otherwise it has to delete the files.

Hard disk defragmenter

A disk defragmenter utility also can be part of a disk repair utility program but it is not primarily concerned with errors. A perfectly functioning disk will, while in use, gradually become less efficient because the constant creation, editing and deletion of files leaves them in a fragmented state. The cause of this is the logical arrangement of data in sectors as discussed in Chapter 3 (Section 3.04), which does not allow a file to be stored as a single block of data.

A simple illustration of the problem is shown in Figure 8.01. Initially file A occupies three sectors fully and part of a fourth one. File B is small so occupies only part of a sector. File C occupies two sectors fully and part of a third. When File B is deleted, the sector remains unfilled because it would require too much system CPU time to rearrange the file organisation every time there is a change. When File A is extended it completely fills the first four sectors and the remainder of the extended file is stored in all of Sector 8 and part of Sector 9. Sector 4 will only be used again if a small file is created or if the disk fills up, when it might store the first part of a longer file.

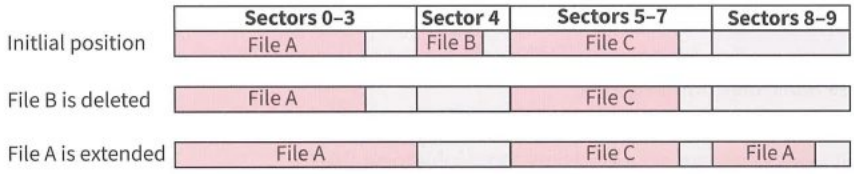


Figure 8.01 File fragmentation on a hard disk

A defragmenter utility program reorganises the file storage to return it to a state where all files are stored in one block across a sequence of sectors. For a large disk this will take some time. It will be impossible if the disk is too full because of the lack of working space for the rearrangement.

TASK 8.02

If you have never used a disk defragmenter or disk repair utility program, can you get access to a system where you can use one? If so, note the changes that are carried out and recorded by the utility program.

Backup software

It is quite likely that you perform a manual backup of your own files every now and then using a flash memory stick. However, an easier way to perform backup is to use a backup utility program. Such a program will:

- establish a schedule for backups
- only create a new backup file when there has been a change.

Question 8.02

In the systems that you use, the technical staff will have made provision for backup. Can you find out what procedures are followed and what hardware is used for this?

File compression

A file compression utility program can be used regularly by an operating system to minimise hard disk storage requirements. Whether or not the operating system does this, a user can still choose to install a suitable program. However, as was discussed in Chapter 1 (Section 1.07) compression is most important when transmitting data. In particular, it makes sense to compress (or 'zip') a file before attaching it to an email.

Virus checker

A virus-checking program should be installed as a permanent facility to protect a computer system. In an ideal world, it would only need to be used to scan a file when the file initially entered the system. Unfortunately, this ideal state can never be realised. When a new virus comes along there is a delay before it is recognised and a further delay before a virus checker has been updated to deal with it. As a result, it is necessary for a virus checker to be regularly updated and for it to scan all files on a computer system as a matter of routine.

8.04 Program libraries

The 'programs' in a program library are usually subroutines created to carry out particular tasks. A programmer can use these within their own programs.

All newly developed programs are likely to contain errors, which only become apparent as the programs are tested or used. It saves a programmer a lot of time and trouble to be able to include already tried and tested subroutines taken from a program library.

The most obvious examples of library routines are the built-in functions available for use when programming in a particular language. Examples of these are discussed in Chapter 14 (Section 14.07). Another example is the collection of over 1600 procedures for mathematical and statistics processing available from the Numerical Algorithms Group (NAG) library. This organisation has been creating routines since 1971 and they are universally accepted as being as reliable as software ever can be.

In Section 8.05, we will discuss the methods available for translation of source code. For now, we simply need an overview of what happens. The source code is written in a programming language of choice. If a compiler is used for the translation and no errors are found, the compiler produces object code (machine code). This code cannot be executed by itself. Instead it has to be linked with the code for any subroutines used by it. It is possible to carry out the linking before loading the full code into memory and running it.

There is a major disadvantage in linking library routines into the executable code. This is because every program using a routine has to have its own copy. This increases the storage space requirement for the executable file. It also increases memory usage when more than one process uses the routine.

The alternative is to use a routine from a dynamic linked library (DLL). When a DLL routine is available the executable code just requires a small piece of code to be included. This allows it to link to the routine, which is stored separately in memory, when execution of the program needs it. Many processes can be linked to the same routine. This has the advantage that the executable files for all programs need less storage space. Memory requirement is also minimised. Another advantage is that if a new version of the routine becomes available it can be loaded into memory so that any program using it is automatically upgraded.

The main disadvantage of using a DLL is that the program is relying on the routine being available and performing the expected function. If for some reason the DLL becomes corrupted or a new version has bugs not yet discovered the program will fail or produce an erroneous result. The user running the program will find it difficult to establish what needs to be done to get the program to run without error.

8.05 Language translators

The use of an assembler for translating a program written in assembly language has been discussed in Chapter 6 (Sections 6.02, 6.03 & 6.04). This chapter will introduce the translators that are used to translate a program written in a high-level procedural language.

Compilers and interpreters

The starting point for using either a compiler or an interpreter is a file containing source code, which is a program written in a high-level language.

For an interpreter the following steps apply.

1. The interpreter program, the source code file and the data to be used by the source code program are all made available.
2. The interpreter program begins execution.
3. The first line of the source code is read.
4. The line is analysed.
5. If an error is found, this is reported and the interpreter program halts execution.
6. If no error is found, the line of source code is converted to an intermediate code.
7. The interpreter program uses this intermediate code to execute the required action.
8. The next line of source code is read and Steps 4–8 are repeated.

For a compiler the following steps apply.

1. The compiler program and the source code file are made available but no data is needed.
2. The compiler program begins execution.
3. The first line of the source code is read.
4. The line is analysed.
5. If an error is found this is recorded.